

Operating System Projects: Linking Theory, Practice and Use

Sub Ramakrishnan, Ann-Marie Lancaster

Bowling Green State University
Department of Computer Science
Bowling Green, Ohio 43403
phone: (419) 372 2337 fax: 419 372 8061
email: CS408@truth.bgsu.edu

Operating Systems (OS) is an essential course in computer science education. In this paper, we describe three course projects which use UNIX® as their basis. We discuss the objectives underlying the design of these projects, and the ways in which these projects provide students with links among theory, practice and use of OS.

1 Introduction

In Operating Systems (OS) courses, providing students with opportunities to engage in practical projects is essential for grasping the underlying theoretical concepts [12, 13]. Project environments in operating system courses have taken many forms. The use of simulated machines as the basis for students projects has spanned over a decade [13, 14]. More recently, instructional systems have been developed which simulate and graphically display the actions of an operating system and allow students to change parameters and modify algorithms [1, 4]. Additionally, the widespread availability of low cost computer systems has provided environments in which students can develop simple, complete operating systems [7, 7, 10] as well as environments in which students can study and modify simple, but fully functional, operating systems [2, 6, 12].

In this paper, we discuss three projects used recently in our OS course. The projects use UNIX as their basis. The first project makes use of UNIX system calls, the second project involves implementation of a UNIX command with which the students are familiar as users, and the third project involves actual modification of UNIX source code. The objectives of these projects are three-fold: (1) The project experience should clearly integrate with class presentations and discussions, enrich students' understanding of these concepts, and enhance their appreciation for the theory which provides the framework for implementation. (2) The project experience should allow students to see the underlying complexities of OS and help them gain insight into what they may encounter if

their future studies or work is in the field of OS. (3) The project experience should provide students a context to understand more clearly their experiences as users of OS.

Although UNIX is a complex tool, its popularity cannot be over emphasized. It is in widespread use in most universities. Students in our OS course are already familiar with UNIX as users which constitutes a strong motivational factor when they begin work on the course projects. Another factor which has influenced our choice is the fact that UNIX principles are extensively documented in the literature. While special purpose OS designed for classroom use are often tied to specific textbooks, many OS textbooks use the UNIX philosophy to illustrate concepts.

Of the several UNIX implementations available today, we chose SunOS®, implemented on SUN workstations. We chose SunOS simply because we have a number of SUN workstations on campus; all of which support SunOS. However, the projects described in this paper will work, without modification, on platforms that run UNIX 4.2/4.3 bsd or *any* of its derivatives (Xenix®, DEC ultrix®, SunOS, and others). There is no functional distinction between these different flavors of UNIX, as they relate to the work described here; thus for the purpose of this discussion, we denote the operating system by the generic name UNIX.

2 The OS Course

Our operating systems course (CS 408) is a one-semester course in which both seniors and some graduate students enroll. The students complete the prerequisites for CS 408, in this order: CS 101 - Introduction to (Pascal) programming, CS 205 - Data Structures using Pascal, CS 207 -Systems Programming, and CS 208 - Advanced Systems Programming. In CS 207, students are introduced to the C programming language, and program extensively in Vax-11 assembler. In CS 208 they design an assembler, loader and interpreter for a hypothetical machine and implement these using the C programming language. Although the hypothetical machine used varies from semester to semester, the machine typically has anywhere from 8 to 16 instructions. Some are single address instructions while others are memory reference instructions.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

ACM-24thCSE-2/93-IN,USA

© 1993 ACM 0-89791-566-6/93/0002/0256...\$1.50

The students are also introduced to interrupt processing and DMA transfer in CS 208. After completing the systems programming sequence, students understand the use of data structures in complex programs, are competent in writing fairly involved programs in C, appreciate the complexities of systems software, and understand how machines execute user programs. Further, students are familiar with the VAX environment. At this point, students have the necessary background to enroll in CS 408; however, most students take other core computer science courses in their junior year, and sign up for CS 408 in their senior year.

In the CS 408 course, students study both the theory and structure of OS. Following an overview of OS, the students are introduced to the concept of a process, process state transition model, relevant process attributes that are maintained by the OS, scheduling algorithms and coordination among processes. These topics are covered during the first five weeks of classes and are the focus of the course projects. The course projects are implemented in the C programming language. Other theory topics covered in CS 408 include memory management, I/O management, device management, and file management.

3 Project Description

In this section, we describe three OS projects. The first two were assigned within the same semester. The third project was used in a previous semester together with a project very similar to the second project.

Project I: Concurrency

In this project, students are asked to implement a solution for the producer-consumer problem. The project is divided into two parts and each part is assigned, submitted and graded separately. The students work in teams of either two or three and a team must complete the first part before proceeding to the second.

1. Uncoordinated Producer-Consumer Problem This phase is very simple and process coordination is *not* employed. The problem proceeds as follows: (a) Initialize the environment by creating a shared memory area where the number of items to be consumed is stored. (b) Accept a count of the maximum number of items to be produced. (c) Fork the process into two processes: producer (or child) and consumer (or parent). (d) During each repetition, the producer reads the shared memory value, increments it by one to signify that an item is produced, and writes it back. (e) During each repetition, the consumer process reads the shared memory value, decrements it by one to signify that an item is consumed, and writes it back. At the end, counts for the number of items produced, the number of items consumed, and the final value of the shared memory value are displayed. The students are excited to see that different runs produce different results!

The project requires the use of the following system calls: (i) *fork* - fork the main program to create

producer and consumer processes; (ii) *shmget* - creates a shared memory segment and returns an id; (iii) *shmat* - attaches the shared memory to the program and returns a pointer to the shared memory; (iv) *sscanf* - read from shared memory; (v) *sprintf* - write to shared memory; (vi) *shmdt* - detach the shared memory from the program; (vi) *shmctl* - destroy the shared memory. The same primitives *sscanf* and *sprintf* used for UNIX file access are also used for shared memory access.

2. Coordinated Producer-Consumer Problem The second part of the problem extends the first part. The difference is that, in this phase, process coordination is employed by accessing the shared memory in a *mutually exclusive* manner. Specifically, during the initialization step (a) when the shared memory is created, a semaphore is also created with an initial value of 0. Step (d) of the first part is modified by wrapping the shared memory access with a P and V operation. Similar modifications are made to step (e) of part 1. Just prior to the completion of execution, the semaphore and the shared memory space are released to the OS. The program submitted by a student is given in Appendix A (for brevity, the header definitions and error checks are not shown). The students spent about 20 hours to complete the two parts.

This project is different from that reported in [6] in that it does not provide a clean interface by encapsulating the system details. The real-world, where each system has its own intricacies, is not ideal and we believe that hiding the system details by a clean interface does not provide the students with a realistic challenge. A classroom discussion covering the basics of the system details was adequate to prepare students for this challenge. Initially, students felt that UNIX documentation was hard to understand. Once the students were past this initial hurdle, their enthusiasm for the project as well as their self-confidence increased. Students were excited to learn the power of *fork* system call. Arbitrary interleaving of messages from the two processes reinforced their comprehension of concurrent programs. Their mastery of the subject was evident when 90% of them gave a correct solution to a non-trivial exam problem on concurrency.

Project II: Process Monitoring

In this project, the students navigate through SunOS system structures to display pertinent information on all memory resident processes (i.e. the UNIX system "ps" command). When they were assigned this project, students were highly skeptical about their ability to implement this function within the time constraints of a semester. Documentation was available to the students in standard textbooks (see [8], page 72-75) as well as on line in UNIX header files.

The students knew from class discussions on process management that for each process the system

maintains two structures: "p area" and the "u area" which together hold all the process related information. We provided them information on the system file /vmunix which holds pointers for various system structures and the system file /dev/kmem which contains the structures themselves. Once provided with information on these UNIX implementation details, the students were able to proceed with the project. They carried out three simple steps (Figure 1). (1) From /vmunix, read the address corresponding to the string "proc". This location holds the pointer to an address in /dev/kmem. (2) From /dev/kmem file, read the content of this address which is the start of the first p area. (3) Read the p area structure and the corresponding u area structure. As shown in Figure 1, the u area can be read through a pointer in the p area. Since the p areas are contiguous, step 3 is repeated for each process by suitably updating the p area pointer.

The resulting program was functionally (about 60%) comparable to the system routine "ps". The students were really excited about their understanding of UNIX internals and being able to access directly the kmem and vmunix system files. Fortunately, the UNIX internal implementation details are sufficiently documented in textbooks that many "hard" projects can indeed be solved by our students! Although we do not include the project code in this paper, the code is about 110 lines and is available from the authors. The students spent an average of about 15 hours to complete the work.

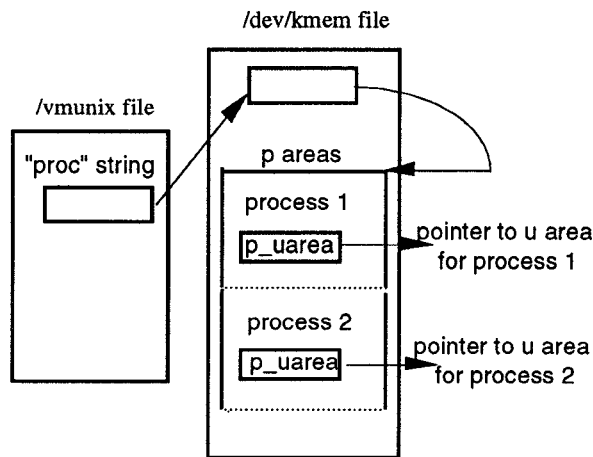


Figure 1. Organization of SunOS Process Structures

Project III: Assigning Process Priorities

In this project, students were asked to "tinker" with the scheduling algorithms which assigned priorities to processes, and to determine in what ways changes to the algorithm affected system performance. It involves modifying UNIX source code (scheduling algorithms) and making the kernel to test the solution. We assume sites have UNIX source code license. We used, inexpensive, Microvax 2000 machines for this project.

To help in our discussion in this paper, we briefly explain how process priorities are calculated; please see page 87-88 of [8] for a more complete explanation of priority calculation. In UNIX, low process priority implies higher actual execution priority. Briefly, process priority, a function of p_cpu , is computed every T_1 time units. p_cpu depends on "system load" and is recomputed every T_0 units. In general, the value of p_cpu tends to decrease every time it is calculated; this decrease helps to decrease process priority or increase actual execution priority. However, for blocked processes (processes blocked for an event for more than T_2 units), p_cpu is computed only when they become unblocked and using a *different* expression. This expression provides for a much more rapid decrease in p_cpu depending on how long the process was blocked. This has the effect of improving the response time for interactive process as compared to CPU intensive (batch) processes. In Unix, $T_0 = 10$ ms, $T_1 = 4 * T_0$, and $T_2 = 1000$ ms. Now one can tinker with any of these quantities: T_0 , T_1 , T_2 , and the expressions for computing process priority and p_cpu (for the two types of processes).

This project required extensive preparation. Fortunately, many of our graduate students are eager to explore UNIX and they produced the documentation for the project and wrote useful comments on the UNIX source files used for scheduling. Students were given documentation on the relevant UNIX procedures (e.g. *timeout*, *hardclock*, *softclock*, *roundrobin*, *schedcpu*, *setpri*; please see page 88-89 [8] under "Process-Priority Routines" for an explanation of these routines) and copies of source code. They were given descriptions of pertinent variables and constants. There was also extensive class discussion of UNIX scheduling algorithms. Finally, they were also given detailed instructions on how to remake the UNIX kernel.

A manufacturer supplied *makefile* contains the necessary commands to "make" or generate the kernel. For tinkering with scheduling algorithms, the students need to modify routines in two source files (*kern_synch.c* and *kern_clock.c*). We copied the *makefile* and these two source files to the class "library"; the students copy them from the library to their account. (We also edited the "path" in the *makefile* so that references to *kern_synch.c* and *kern_clock.c* are to student copies and *not* to the original ones.) Then, they make the required modification to *kern_synch.c* and/or *kern_clock.c* and invoke the *makefile*. The newly generated kernel can be tested by booting the system using the command `-boot/2`; `/2` option prompts for the name of the new kernel.

Students worked in teams of two or three and the instructor met with each team once or twice during the project period to stimulate their thinking on how to discover and construct their experiments and test situations. At the end of the project period, each team demonstrated the highlights of their experiments. The aspects of the scheduling algorithms that were the focus of their experiments varied among the teams.

The following are suggested modifications for experimenting with the scheduling algorithm. (1) Simulate a FIFO scheduling algorithm. Set T2 to a very high value and also ensure all processes have equal priority. Since context switching occurs infrequently, processes are completed in a FIFO manner, and the standard roundrobin scheduler does not kick in. (2) Favor batch processes over interactive processes by modifying the expression for computing *p_cpu* for blocked processes, i.e. allow *p_cpu* to decrease less rapidly. In this case, the performance of a process that has plenty of read/write degrades noticeably. (3) Set T0 to 0.1 ms instead of 10 ms. Then, *p_cpu* is calculated much more frequently. With T0 set to 0.1 ms, the roundrobin scheduling algorithm kicks in more frequently resulting in too many context switches. The elapsed time for a process is significantly more than if the original value of T0 is maintained.

4 Extensions

The projects described in the previous section can be extended in a number of different ways. We elaborate on some possible extensions here. The producer consumer project can be generalized to the "n readers and m writers" problem which is discussed in any standard text (see for example, [11], page 157-158). Herein, multiple readers may be in the critical section at the same time. Each writer process, every time through its loop, writes "data" to a shared buffer (manipulated using shared memory calls similar to Project I above). Each reader reads the "data" from this buffer and displays on the terminal: readerID, "data". The student implementation is correct if each reader displays all data, in the order the writers write to the buffer and all processes terminate gracefully at the very end.

In this case, the algorithm requires the use of two semaphores: "A" and "B". "A" is shared between the readers. The writer coordinates writing the "data" to the buffer using "A". Similarly, "A" is used by the first reader to ensure that no writer is updating the buffer, and by the last reader to release the critical section. Semaphore "B" is used among the readers to maintain a correct count of the number of readers in the critical section. The count can be kept in shared memory, or semaphore "B" can be implemented as a counting semaphore (whose value ranges from 0 to n).

The mechanism for the writer to obtain the "data" (console input or randomly generated), spawning the reader and writer processes (a main program accepts system parameters and spawns all the processes by means of "fork" system call), implementing the "data" buffer (linear or circular buffer), terminating the program (for example, "S" signals end of "data"), deciding process priority ("weak" reader priority, "strong writer" priority) are some of the things that can be specified by the instructor or determined by the student to generate some interesting variations to this basic theme. We included one such variation in our operating systems course. Student feedback was positive

and they felt that but for this project they would never have fully understood the internal details of this algorithm.

Another possible extension to the producer consumer project is illustrated by an interesting assignment recently posted to a newsgroup [9]. This assignment, *conference application*, was a class project for the operating systems course at Columbia University. The program takes as argument the nickname of the user and the name of the conference (allows multiple conferences on the same machine). Any number of users may join the conference. Whatever users type on their terminals should appear on everyone's screen prefixed by sender's name. It is clear that shared memory and semaphores are the essential concepts to solve this problem.

Finally, the project on process monitoring can be extended to include not only memory resident processes but also swapped out processes. The solution to this problem is straight forward. The u area for a swapped out process is disk resident, and information indicating whether or not a process is swapped out is available in the p area. The address in secondary storage where the contents of the u area are stored is also available in the p area. Retrieving contents of the u area from disk provides opportunities for students to learn about disk file manipulation primitives (open, read, seek, write, close).

5 Concluding Remarks

In this paper, we described some projects used recently in our operating system course. Each of these projects used an actual operating system as its base, UNIX. We are pleased with the outcome and expect to continue using this approach. In particular, we think that the projects provide students with the means to link theory, practice and use. Although the students were apprehensive when they were assigned a project, we gave them sufficient documentation as well as personal instruction and assistance to put them at ease. Once they gained a certain amount of self-confidence and momentum, they continued with the project with a great deal of enthusiasm. After completing the projects, students not only felt that they understood the concepts presented in class more clearly, but also that they were much more knowledgeable computer users. They were particularly delighted because UNIX was no longer the nebulous and mysterious entity they perceived it to be.

We also discussed several extensions to these projects. Together, these problems provide opportunities for students to engage in practical operating system projects. Note that the projects described in this paper will work, without modification, on platforms that run UNIX 4.2/4.3 bsd or *any* of its derivatives (Xenix, DEC ultrix, SunOS, and others). This is because basic kernel data structures are preserved across these variants. We also found that the concepts learned in CS 208 relating to the full implementation of complex system software modules (e.g. assembler, loader) were not really essential to the students' performance in the operating systems course.

Instead, understanding the concepts of interrupt processing and experience on programming non-trivial C problems (using pointers, linked list) appeared to be the essential prerequisites for undertaking the CS 408 projects.

References

- [1] Canas, D. A., "Graphos: A Graphic Operating System," ACM 18th SIGCSE Proceedings, February 1987, p. 201.
- [2] Comer, D. and Munson, S., Operating Systems Design: I: The XINU Approach, Prentice-Hall, 1989.
- [3] Donaldson, J. L., "Operating Systems from Assembler to C," ACM 21st SIGCSE Proceedings, February 1990, p.121.
- [4] Hays J et. al., "Simulation of Process and Resource Management in a Multiprogramming Operating System", ACM 21st SIGCSE Proceedings, February 1990, p. 125.
- [5] Higginbotham, C. and Morelli, R., "A System for Teaching Concurrent Programming" , ACM 22nd SIGCSE Proceedings, March 1991, p. 309.
- [6] Hummel, J., "Xinu/WU: An Improved PC-Xinu Clone?", ACM 20th SIGCSE Proceedings, 1989, p. 226.
- [7] Lane, M. and Ghosal, A., "MPX-PC: An Operating System Project for the PC", ACM 20th SIGCSE Proceedings, 1989, p. 231.
- [8] Leffler, S et. al., The Design and Implementation of the 4.3BSD UNIX Operating System, Addison-Wesley Publishing Company, 1989.
- [9] K. Levy, "Operating Systems Project Notes", posted to [comp.unix.solaris](mailto:comp.unix.solaris@news.columbia.edu) newsgroup, article # 1992 Jun15.183906.10540@news.columbia.edu.
- [10] Reek, M., "An Undergraduate Operating Systems Lab Course", ACM 21st SIGCSE Proceedings, February 1990, p. 171.
- [11] Silberschatz, A. and Peterson, J., Operating Systems Concepts, third edition, Addison-Wesley Publishing Company, 1991.
- [12] Tanenbaum, A., Operating Systems: Design and Implementation, Prentice Hall Inc., 1987.
- [13] Walker, J. and Hughes, C., "POPSS- A Parametric Operating Systems Simulator," ACM 4th SIGCSE Proceedings, Feb. 1973, p. 166.
- [14] Wolfe, J., "Operating System Projects on Two Simulated Machines", ACM 18th SIGCSE Proceedings, February 1987, p. 212.

```
#define key 1003 /* some random number */
```

```
#define shmall 0666 /* permissions */
#define semall 0666
#define nsops 1 /* # semaphores needed */
struct sembuf semary; /* semaphore structure */
union senum {
    int val; struct SemId_ds *buf; ushort *array; } arg;
int produce=0, consume=0,maximum, process,count;
int semaphore,ShmFlag,SemFlag,ShmId,SemId;
char *shmad1,*shmad2;
main () {
    Initialize();
    printf("Produce #? "); scanf("%d",&maximum);
    if ((process=fork()) == 0) /* child */
        { while (produce < maximum) {
            Wait("Child P operation");
            sscanf(shmad1,"%d",&count);
            produce++; count++;
            printf(shmad1,"%d",count);
            Signal("Child V operation"); } /* end while */
        exit(); } /* end of child */
    else /* parent processes */
        { while (consume < maximum) {
            Wait("Parent P operation");
            sscanf(shmad1,"%d",&count);
            if (count > 0 ) { count--; consume++;
                sprintf(shmad1,"%d", count); }
            Signal("Parent V operation"); } /* end of while */
        wait(NULL); /* wait for child to end */
        Terminate(); } /* end of parent */
    } /* end of main program */
Initialize()
    ShmFlag=IPC_CREAT | shmall;
    SemFlag=IPC_CREAT | semall; shmad2=0;
    ShmId=(shmget((key_t)key,32,ShmFlag);
    shmad1=shmat(ShmId,shmad2,ShmFlag);
    SemId=(semget((key_t)key,1,SemFlag);
    arg.val=1; semary.sem_num=0;
    semary.sem_flg = semall;
    semctl(SemId,0,SETVAL,arg); } /* end initialize */
Terminate()
    shmdt(shmad1);
    shmctl(ShmId,IPC_RMID,shmad1);
    semctl(SemId,1,IPC_RMID); } /* end terminate */
Signal(message) char *message;
{ char buf[100]; int semaphore;
  sprintf(buf, " %s", message);
  semary.sem_op=1;
  semop(SemId,semary,1); } /* end signal */
Wait(message)
{ char buf[100]; int semaphore;
  sprintf(buf, " %s", message);
  semary.sem_op= -1; /* decrement */
  semop(SemId,semary,1); } /* end wait */
```

Appendix A